

DevOps Project README

Complete Deployment Guide — Steps 1–7

This PDF is a formatted implementation guide based on the instructions previously provided from the uploaded project PDF. It preserves the source project's terminology, configuration, commands, and deployment sequence. The source PDF ends at Step 6.4; Step 7 below is explicitly a final verification checklist rather than a source-derived numbered step.

Table of Contents

- Prerequisites
- Step 1 — Local Setup & Application Validation
- Step 2 — Containerization & Local Testing
- Step 3 — Infrastructure Provisioning with Terraform
- Step 4 — Jenkins Setup on AWS
- Step 5 — Create the CI/CD Pipeline
- Step 6 — Deploy Application & Validate
- Step 7 — Final Verification
- Useful Terraform and Docker Commands
- Deployment Configuration Summary
- Complete Project Checklist

Prerequisites

- Docker
- AWS CLI
- Node.js & NPM
- Terraform
- GitHub account with SSH configured
- AWS account/IAM credentials
- GitHub Personal Access Token for Jenkins
- EC2 key pair

Note: The source PDF specifies the first five prerequisites. The remaining items are required by the deployment instructions themselves.

Step 1 — Local Setup & Application Validation

1.1 Clone the Repository

```
git clone https://github.com/TayoLusi19/devops-code-challenge1.git
cd devops-code-challenge1
```

1.2 Install Backend Dependencies

```
cd backend
npm ci
```

1.3 Start the Backend

```
npm start
```

Backend: <http://localhost:8080>

1.4 Configure the Frontend

Edit frontend/src/config.js:

```
const config = {
  backendUrl: "http://localhost:8080",
};

export default config;
```

1.5 Install Frontend Dependencies

```
cd ../frontend
npm ci
```

1.6 Start the Frontend

```
npm start
```

Frontend: <http://localhost:3000>

Verify the application displays: SUCCESS + GUID

Step 2 — Containerization & Local Testing

2.1 Backend Dockerfile

```
FROM node:16
```

```
WORKDIR /app

COPY package*.json ./

RUN npm ci

COPY . .

EXPOSE 8080

CMD ["npm", "start"]
```

2.2 Frontend Dockerfile

```
FROM node:18

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

RUN npm run build

RUN npm install -g serve

EXPOSE 3000

CMD ["serve", "-s", "build"]
```

2.3 Build Backend

```
docker build -t backend-app .
```

2.4 Configure Frontend for Docker

```
const config = {
  backendUrl: "http://host.docker.internal:8080",
};

export default config;
```

2.5 Build Frontend

```
docker build -t frontend-app ./frontend
```

2.6 Run Backend

```
docker run -d \
  --name backend \
  -p 8080:8080 \
  backend-app
```

2.7 Run Frontend

```
docker run -d \
  --name frontend \
  -p 3000:3000 \
  frontend-app
```

2.8 Verify

Open <http://localhost:8080> and <http://localhost:3000>. Verify SUCCESS + GUID.

2.9 Push Changes

```
git add .
git commit -m "Add Dockerfiles and updated project files for ECS deployment"
git push origin main
```

Step 3 — Infrastructure Provisioning with Terraform

Create a terraform/ directory. The following configurations are the implementation snippets previously provided from the source.

3.1 provider.tf

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "6.0.0-beta2"
    }
  }
}

provider "aws" {
  region = var.aws_region
}
```

3.2 networking.tf

```
resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_hostnames = true
  enable_dns_support   = true

  tags = {
    Name        = "${var.project_name}-vpc"
    Environment = var.environment
  }
}

resource "aws_subnet" "public" {
  count = 2
  vpc_id = aws_vpc.main.id

  cidr_block = cidrsubnet(var.vpc_cidr, 8, count.index)

  availability_zone = count.index == 0 ? "us-east-2a" : "us-east-2b"
  map_public_ip_on_launch = true

  tags = {
    Name        = "${var.project_name}-public-subnet-${count.index + 1}"
    Environment = var.environment
  }
}

resource "aws_subnet" "private" {
  count = 2
  vpc_id = aws_vpc.main.id

  cidr_block = cidrsubnet(var.vpc_cidr, 8, count.index + 2)

  availability_zone = count.index == 0 ? "us-east-2a" : "us-east-2b"

  tags = {
    Name        = "${var.project_name}-private-subnet-${count.index + 1}"
    Environment = var.environment
  }
}

resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id
}

resource "aws_eip" "nat" {
  domain = "vpc"
}

resource "aws_nat_gateway" "main" {
  allocation_id = aws_eip.nat.id
  subnet_id     = aws_subnet.public[0].id
}

resource "aws_route_table" "public" {
```

```

vpc_id = aws_vpc.main.id

route {
  cidr_block = "0.0.0.0/0"
  gateway_id = aws_internet_gateway.main.id
}

resource "aws_route_table" "private" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block      = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.main.id
  }
}

resource "aws_route_table_association" "public" {
  count = 2
  subnet_id = aws_subnet.public[count.index].id
  route_table_id = aws_route_table.public.id
}

resource "aws_route_table_association" "private" {
  count = 2
  subnet_id = aws_subnet.private[count.index].id
  route_table_id = aws_route_table.private.id
}

```

3.3 ECS/Fargate — Cluster and Logs

```

resource "aws_ecs_cluster" "main" {
  name = "${var.project_name}-cluster"

  setting {
    name = "containerInsights"
    value = "enabled"
  }
}

resource "aws_cloudwatch_log_group" "frontend" {
  name = "/ecs/${var.project_name}-frontend"
  retention_in_days = 7
}

resource "aws_cloudwatch_log_group" "backend" {
  name = "/ecs/${var.project_name}-backend"
  retention_in_days = 7
}

```

3.4 Frontend Task Definition

```

resource "aws_ecs_task_definition" "frontend" {
  family = "${var.project_name}-frontend"
  network_mode = "awsvpc"
  requires_compatibilities = ["FARGATE"]

  cpu = var.frontend_cpu
  memory = var.frontend_memory

  execution_role_arn = aws_iam_role.ecs_task_execution_role.arn
  task_role_arn = aws_iam_role.ecs_task_role.arn

  container_definitions = jsonencode([
    {
      name = "frontend"
      image = var.frontend_image != "" ? var.frontend_image : "${aws_ecr_repository.frontend.repository_url}:latest"

      portMappings = [{
        containerPort = var.frontend_port
        protocol = "tcp"
      }]

      environment = [{
        name = "REACT_APP_BACKEND_URL"
        value = "http://${aws_lb.main.dns_name}"
      }]
    }
  ])
}

```

```

    }]
    logConfiguration = {
      logDriver = "awslogs"
      options = {
        awslogs-group      = aws_cloudwatch_log_group.frontend.name
        awslogs-region     = "us-east-2"
        awslogs-stream-prefix = "ecs"
      }
    }
    essential = true
  }
})
}

```

3.5 Backend Task Definition

```

resource "aws_ecs_task_definition" "backend" {
  family           = "${var.project_name}-backend"
  network_mode     = "awsvpc"
  requires_compatibilities = ["FARGATE"]

  cpu           = var.backend_cpu
  memory        = var.backend_memory

  execution_role_arn = aws_iam_role.ecs_task_execution_role.arn
  task_role_arn      = aws_iam_role.ecs_task_role.arn

  container_definitions = jsonencode([
    {
      name       = "backend"
      image      = var.backend_image != "" ? var.backend_image : "${aws_ecr_repository.backend.repository_url}:latest"

      portMappings = [
        {
          containerPort = var.backend_port
          protocol       = "tcp"
        }
      ]

      environment = [
        {
          name  = "CORS_ORIGIN"
          value = "http://${aws_lb.main.dns_name}"
        }
      ]

      logConfiguration = {
        logDriver = "awslogs"
        options = {
          awslogs-group      = aws_cloudwatch_log_group.backend.name
          awslogs-region     = "us-east-2"
          awslogs-stream-prefix = "ecs"
        }
      }

      essential = true
    }
  ])
}

```

3.6 ECS Services

```

resource "aws_ecs_service" "frontend" {
  name           = "${var.project_name}-frontend-service"
  cluster        = aws_ecs_cluster.main.id
  task_definition = aws_ecs_task_definition.frontend.arn

  desired_count = var.desired_tasks
  launch_type   = "FARGATE"

  network_configuration {
    security_groups = [aws_security_group.frontend.id]
    subnets        = aws_subnet.private[*].id
    assign_public_ip = false
  }

  load_balancer {
    target_group_arn = aws_lb_target_group.frontend.arn
  }
}

```

```

        container_name = "frontend"
        container_port = var.frontend_port
    }

    depends_on = [aws_lb_listener.frontend]
}

resource "aws_ecs_service" "backend" {
    name = "${var.project_name}-backend-service"
    cluster = aws_ecs_cluster.main.id
    task_definition = aws_ecs_task_definition.backend.arn

    desired_count = var.desired_tasks
    launch_type = "FARGATE"

    network_configuration {
        security_groups = [aws_security_group.backend.id]
        subnets = aws_subnet.private[*].id
        assign_public_ip = false
    }

    load_balancer {
        target_group_arn = aws_lb_target_group.backend.arn
        container_name = "backend"
        container_port = var.backend_port
    }
}

```

3.7 ECR Repositories

```

resource "aws_ecr_repository" "frontend" {
    name = "${var.project_name}-frontend"
    image_tag_mutability = "MUTABLE"

    image_scanning_configuration {
        scan_on_push = true
    }
}

resource "aws_ecr_repository" "backend" {
    name = "${var.project_name}-backend"
    image_tag_mutability = "MUTABLE"

    image_scanning_configuration {
        scan_on_push = true
    }
}

```

3.8 Security Groups

```

resource "aws_security_group" "alb" {
    name = "${var.project_name}-alb-sg"
    description = "Security group for Application Load Balancer"
    vpc_id = aws_vpc.main.id

    ingress {
        from_port = 80
        to_port = 80
        protocol = "tcp"
        cidr_blocks = ["0.0.0.0/0"]
    }

    ingress {
        from_port = 443
        to_port = 443
        protocol = "tcp"
        cidr_blocks = ["0.0.0.0/0"]
    }

    egress {
        from_port = 0
        to_port = 0
        protocol = "-1"
        cidr_blocks = ["0.0.0.0/0"]
    }
}

```

```

resource "aws_security_group" "frontend" {
  name = "${var.project_name}-frontend-sg"
  vpc_id = aws_vpc.main.id

  ingress {
    from_port = var.frontend_port
    to_port = var.frontend_port
    protocol = "tcp"
    security_groups = [aws_security_group.alb.id]
  }

  egress {
    from_port = 0
    to_port = 0
    protocol = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_security_group" "backend" {
  name = "${var.project_name}-backend-sg"
  vpc_id = aws_vpc.main.id

  ingress {
    from_port = var.backend_port
    to_port = var.backend_port
    protocol = "tcp"
    security_groups = [aws_security_group.frontend.id]
  }

  ingress {
    from_port = var.backend_port
    to_port = var.backend_port
    protocol = "tcp"
    security_groups = [aws_security_group.alb.id]
  }

  egress {
    from_port = 0
    to_port = 0
    protocol = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_security_group" "jenkins" {
  name = "${var.project_name}-jenkins-sg"
  vpc_id = aws_vpc.main.id

  ingress {
    from_port = 80
    to_port = 80
    protocol = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port = 443
    to_port = 443
    protocol = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port = 22
    to_port = 22
    protocol = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port = 8080
    to_port = 8080
    protocol = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

```



```

egress {
  from_port = 0
  to_port   = 0
  protocol  = "-1"
  cidr_blocks = ["0.0.0.0/0"]
}
}

```

Note: The source uses 0.0.0.0/0 for Jenkins SSH and web access. This is reproduced for fidelity to the source. For production, restrict administrative access to trusted IP ranges.

3.9 IAM Roles

```

resource "aws_iam_role" "ecs_task_execution_role" {
  name = "${var.project_name}-ecs-task-execution-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action = "sts:AssumeRole"
      Effect = "Allow"
      Principal = {
        Service = "ecs-tasks.amazonaws.com"
      }
    }]
  })
}

resource "aws_iam_role_policy_attachment" "ecs_task_execution_role_policy" {
  role       = aws_iam_role.ecs_task_execution_role.name
  policy_arn = "arn:aws:iam::aws:policy/service-role/AmazonECSTaskExecutionRolePolicy"
}

resource "aws_iam_role" "ecs_task_role" {
  name = "${var.project_name}-ecs-task-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action = "sts:AssumeRole"
      Effect = "Allow"
      Principal = {
        Service = "ecs-tasks.amazonaws.com"
      }
    }]
  })
}

resource "aws_iam_policy" "ecs_task_logs" {
  name = "${var.project_name}-ecs-task-logs"

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Action = [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents"
      ]
      Resource = "*"
    }]
  })
}

resource "aws_iam_role_policy_attachment" "ecs_task_logs" {
  role       = aws_iam_role.ecs_task_role.name
  policy_arn = aws_iam_policy.ecs_task_logs.arn
}

```

3.10 Application Load Balancer

```

resource "aws_lb" "main" {
  name          = "${var.project_name}-alb"
  internal      = false
  load_balancer_type = "application"
}

```

```

    security_groups    = [aws_security_group.alb.id]
    subnets           = aws_subnet.public[*].id
}

resource "aws_lb_target_group" "frontend" {
    name      = "${var.project_name}-frontend-tg"
    port      = var.frontend_port
    protocol  = "HTTP"
    vpc_id    = aws_vpc.main.id
    target_type = "ip"

    health_check {
        enabled            = true
        healthy_threshold  = 2
        interval           = 30
        matcher            = "200"
        path               = "/"
        port               = "traffic-port"
        protocol           = "HTTP"
        timeout            = 5
        unhealthy_threshold = 2
    }
}

resource "aws_lb_target_group" "backend" {
    name      = "${var.project_name}-backend-tg"
    port      = var.backend_port
    protocol  = "HTTP"
    vpc_id    = aws_vpc.main.id
    target_type = "ip"

    health_check {
        enabled            = true
        healthy_threshold  = 2
        interval           = 30
        matcher            = "200"
        path               = "/health"
        port               = "traffic-port"
        protocol           = "HTTP"
        timeout            = 5
        unhealthy_threshold = 2
    }
}

resource "aws_lb_listener" "frontend" {
    load_balancer_arn = aws_lb.main.arn
    port              = "80"
    protocol           = "HTTP"

    default_action {
        type            = "forward"
        target_group_arn = aws_lb_target_group.frontend.arn
    }
}

resource "aws_lb_listener_rule" "backend" {
    listener_arn = aws_lb_listener.frontend.arn
    priority     = 100

    action {
        type            = "forward"
        target_group_arn = aws_lb_target_group.backend.arn
    }

    condition {
        path_pattern {
            values = ["/api/*"]
        }
    }
}

```

3.11 Auto Scaling

```

resource "aws_appautoscaling_target" "frontend" {
    max_capacity      = var.max_tasks
    min_capacity      = var.min_tasks
    resource_id       = "service/${aws_ecs_cluster.main.name}/${aws_ecs_service.frontend.name}"
}

```

```

    scalable_dimension = "ecs:service:DesiredCount"
    service_namespace = "ecs"
}

resource "aws_appautoscaling_policy" "frontend_cpu" {
  name           = "${var.project_name}-frontend-cpu-policy"
  policy_type     = "TargetTrackingScaling"
  resource_id     = aws_appautoscaling_target.frontend.resource_id
  scalable_dimension = aws_appautoscaling_target.frontend.scalable_dimension
  service_namespace = aws_appautoscaling_target.frontend.service_namespace

  target_tracking_scaling_policy_configuration {
    predefined_metric_specification {
      predefined_metric_type = "ECSServiceAverageCPUUtilization"
    }
    target_value = var.cpu_threshold
  }
}

resource "aws_appautoscaling_target" "backend" {
  max_capacity      = var.max_tasks
  min_capacity      = var.min_tasks
  resource_id       = "service/${aws_ecs_cluster.main.name}/${aws_ecs_service.backend.name}"
  scalable_dimension = "ecs:service:DesiredCount"
  service_namespace = "ecs"
}

resource "aws_appautoscaling_policy" "backend_cpu" {
  name           = "${var.project_name}-backend-cpu-policy"
  policy_type     = "TargetTrackingScaling"
  resource_id     = aws_appautoscaling_target.backend.resource_id
  scalable_dimension = aws_appautoscaling_target.backend.scalable_dimension
  service_namespace = aws_appautoscaling_target.backend.service_namespace

  target_tracking_scaling_policy_configuration {
    predefined_metric_specification {
      predefined_metric_type = "ECSServiceAverageCPUUtilization"
    }
    target_value = var.cpu_threshold
  }
}

```

3.12 Jenkins EC2

```

data "aws_ami" "amazon_linux_2023" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["al2023-ami-*-x86_64"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
}

resource "aws_instance" "jenkins_master" {
  ami           = data.aws_ami.amazon_linux_2023.id
  instance_type = "t3.small"
  key_name      = "1PU"

  vpc_security_group_ids = [aws_security_group.jenkins.id]
  subnet_id              = aws_subnet.public[0].id

  root_block_device {
    volume_size = 30
    volume_type = "gp3"
    encrypted   = true
  }
}

resource "aws_eip" "jenkins_master" {
  instance = aws_instance.jenkins_master.id
  domain   = "vpc"
}

```

```
}
```

Note: If your AWS key pair has a different name, replace `key_name = "1PU"` with your actual key pair name.

3.13 variables.tf

```
variable "aws_region" {
  type    = string
  default = "us-east-2"
}

variable "project_name" {
  type    = string
  default = "devops-challenge"
}

variable "environment" {
  type    = string
  default = "production"
}

variable "vpc_cidr" {
  type    = string
  default = "10.0.0.0/16"
}

variable "frontend_port" {
  type    = number
  default = 3000
}

variable "backend_port" {
  type    = number
  default = 8080
}

variable "frontend_image" {
  type    = string
  default = ""
}

variable "backend_image" {
  type    = string
  default = ""
}

variable "frontend_cpu" {
  type    = number
  default = 512
}

variable "frontend_memory" {
  type    = number
  default = 1024
}

variable "backend_cpu" {
  type    = number
  default = 512
}

variable "backend_memory" {
  type    = number
  default = 1024
}

variable "min_tasks" {
  type    = number
  default = 1
}

variable "desired_tasks" {
  type    = number
  default = 1
}

variable "max_tasks" {
```

```

    type    = number
    default = 4
}

variable "cpu_threshold" {
    type    = number
    default = 50
}

```

3.14 outputs.tf

```

output "vpc_id" {
    value = aws_vpc.main.id
}

output "vpc_cidr" {
    value = aws_vpc.main.cidr_block
}

output "public_subnet_ids" {
    value = aws_subnet.public[*].id
}

output "private_subnet_ids" {
    value = aws_subnet.private[*].id
}

output "frontend_repository_url" {
    value = aws_ecr_repository.frontend.repository_url
}

output "backend_repository_url" {
    value = aws_ecr_repository.backend.repository_url
}

output "alb_dns_name" {
    value = aws_lb.main.dns_name
}

output "ecs_cluster_name" {
    value = aws_ecs_cluster.main.name
}

output "jenkins_master_public_ip" {
    value = aws_eip.jenkins_master.public_ip
}

output "jenkins_url" {
    value = "http://${aws_eip.jenkins_master.public_ip}:8080"
}

```

3.15 Apply Terraform

```

terraform init
terraform plan
terraform apply

# Or:
terraform apply -auto-approve

```

Validate the AWS resources in the console: VPC, subnets, ECS cluster/services, ECR repositories, ALB, Jenkins EC2, security groups, and IAM resources.

```

git add terraform
git commit -m "Updated Terraform Files"
git push origin main

```

Step 4 — Jenkins Setup on AWS

4.1 Update Amazon Linux 2023

```
sudo dnf update
sudo dnf upgrade
```

4.2 Install Docker

```
sudo dnf install docker -y
sudo systemctl start docker
sudo systemctl enable docker
sudo systemctl status docker
```

4.3 Run Jenkins

```
getent group docker

docker run -d \
  --name jenkins \
  -p 8080:8080 \
  -v /var/jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  --group-add <group-number> \
  jenkins/jenkins:lts
```

4.4 Install Docker Inside Jenkins

```
docker exec -u root -it jenkins bash

apt-get update
apt-get install -y docker.io
docker --version

exit
docker restart jenkins
```

4.5 Jenkins Initial Password

```
sudo docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Jenkins URL: http://:8080

4.6 Plugins

- Docker
- Amazon EC2
- Amazon Elastic Container Service (ECS) / Fargate
- AWS CLI

4.7 Credentials

GitHub: Manage Jenkins → Credentials → (global) → Add credentials → Username with password. Use your GitHub username and PAT. AWS: add an AWS Credentials entry containing the IAM access ID and secret access key.

Step 5 — Create the CI/CD Pipeline

5.1 Jenkinsfile

```
pipeline {
  agent any

  environment {
    AWS_REGION = 'your-aws-region'
    FRONTEND_REPO = 'your-frontend-ecr-repo-uri'
    BACKEND_REPO = 'your-backend-ecr-repo-uri'
  }
}
```

```

}

stages {
  stage('Checkout code') {
    steps {
      checkout scm
    }
  }

  stage('Build Docker images') {
    steps {
      script {
        sh 'docker build -t frontend:latest ./frontend'
        sh 'docker build -t backend:latest ./backend'
      }
    }
  }

  stage('Authenticate to ECR') {
    steps {
      withCredentials([
        [
          $class: 'AmazonWebServicesCredentialsBinding',
          credentialsId: 'your-aws-credentials-id'
        ]
      ]) {
        script {
          sh '''
            aws --version

            aws ecr get-login-password --region $AWS_REGION |
            docker login --username AWS --password-st

            aws ecr get-login-password --region $AWS_REGION |
            docker login --username AWS --password-st

            ...
          '''
        }
      }
    }
  }

  stage('Tag and Push images to ECR') {
    steps {
      script {
        sh '''
          docker tag frontend:latest $FRONTEND_REPO:latest
          docker tag backend:latest $BACKEND_REPO:latest
          docker push $FRONTEND_REPO:latest
          docker push $BACKEND_REPO:latest

          ...
        '''
      }
    }
  }

  stage('Update ECS services') {
    steps {
      withCredentials([
        [
          $class: 'AmazonWebServicesCredentialsBinding',
          credentialsId: 'your-aws-credentials-id'
        ]
      ]) {
        script {
          sh '''
            aws ecs update-service --cluster your-ecs-cluster-name

            aws ecs update-service --cluster your-ecs-cluster-name

            ...
          '''
        }
      }
    }
  }
}

post {
  always {
    cleanWs()
  }
}

```

```
}  
}  
}
```

5.2 Replace Jenkinsfile Placeholders

```
terraform output frontend_repository_url  
terraform output backend_repository_url  
terraform output ecs_cluster_name
```

Set AWS_REGION to us-east-2. Set the ECR repository variables to the Terraform outputs. Set the AWS credential ID to the Jenkins credential you created. With project_name=devops-challenge, the service names are devops-challenge-frontend-service and devops-challenge-backend-service.

5.3 Push Jenkinsfile

```
git add Jenkinsfile  
git commit -m "Jenkinsfile for CI/CD Pipeline"  
git push origin main
```

5.4 Configure Jenkins

- Create a Jenkins Pipeline job.
- Select Pipeline from SCM.
- Point it to the GitHub repository.
- Configure a GitHub webhook or polling.
- For a webhook, use `http://:8080/github-webhook/` with `application/json`.
- Run the initial build.

Step 6 — Deploy Application & Validate

6.1 Trigger Jenkins Pipeline

- Code checkout succeeds
- Docker images build successfully
- ECR authentication succeeds
- Images are pushed
- ECS services are updated

6.2 Check ECR

Verify frontend and backend repositories contain the updated images.

6.3 Check ECS

Verify frontend and backend services are running tasks using the latest images.

6.4 Update Backend Configuration

The source instructs you to update backend config.js so it uses the Application Load Balancer instead of the localhost Docker container. The source does not provide the exact replacement configuration value in the final extracted page.

```
terraform output alb_dns_name
```

Step 7 — Final Verification

Note: The uploaded source PDF does not contain a numbered Step 7. This section is a final verification checklist derived from the deployment requirements already described.

- Jenkins pipeline completes successfully.
- Frontend and backend ECR images are updated.
- Frontend ECS service is running.
- Backend ECS service is running.
- ALB target groups are healthy.
- The application is accessible through the ALB DNS name.
- The /api/* path routes to the backend.
- The application returns the expected SUCCESS + GUID result.

Useful Terraform Commands

```
terraform init
terraform validate
terraform plan
terraform apply
terraform apply -auto-approve
terraform output

terraform output alb_dns_name
terraform output ecs_cluster_name
terraform output frontend_repository_url
terraform output backend_repository_url
terraform output jenkins_master_public_ip
terraform output jenkins_url
```

Useful Docker Commands

```
docker ps
docker ps -a
docker images

docker stop frontend
docker stop backend

docker rm frontend
docker rm backend

docker logs backend
docker logs frontend
```

Deployment Configuration Summary

Component	Configuration
AWS Region	us-east-2
Project	devops-challenge
Environment	production
VPC CIDR	10.0.0.0/16
Frontend	Port 3000
Backend	Port 8080
ECS	Fargate
Frontend CPU / Memory	512 / 1024 MB
Backend CPU / Memory	512 / 1024 MB
Tasks	Min 1 / Desired 1 / Max 4
Auto Scaling	50% CPU
ALB	HTTP port 80
Frontend Health Check	/
Backend Health Check	/health
Backend Route	/api/*
Jenkins	Port 8080
Jenkins EC2	t3.small
Jenkins Storage	30 GB gp3
ECR Scanning	Enabled
ECR Lifecycle	Keep last 5 v* images

Complete Project Checklist

Step 1 — Local Setup

- ■ Install prerequisites
- ■ Clone GitHub repository
- ■ Install backend dependencies
- ■ Start backend

- ■ Configure frontend
- ■ Install frontend dependencies
- ■ Start frontend
- ■ Verify SUCCESS + GUID

Step 2 — Docker

- ■ Create backend Dockerfile
- ■ Create frontend Dockerfile
- ■ Build backend image
- ■ Configure Docker backend URL
- ■ Build frontend image
- ■ Run containers
- ■ Verify communication
- ■ Push changes

Step 3 — AWS/Terraform

- ■ Create networking
- ■ Create ECS/Fargate resources
- ■ Create ECR
- ■ Create security groups
- ■ Create IAM roles
- ■ Create ALB
- ■ Configure autoscaling
- ■ Create Jenkins EC2
- ■ Create variables and outputs
- ■ terraform init / plan / apply
- ■ Validate AWS resources

Step 4 — Jenkins

- ■ Update Amazon Linux
- ■ Install Docker
- ■ Run Jenkins
- ■ Install Docker inside Jenkins
- ■ Retrieve initial password
- ■ Install plugins
- ■ Configure GitHub PAT
- ■ Configure AWS credentials

Step 5 — CI/CD

- ■ Create Jenkinsfile
- ■ Replace placeholders
- ■ Push Jenkinsfile
- ■ Create Jenkins Pipeline
- ■ Configure SCM
- ■ Configure webhook/polling
- ■ Run initial build

Step 6 — Deployment

- ■ Trigger pipeline
- ■ Verify ECR images
- ■ Verify ECS services
- ■ Update backend configuration

Step 7 — Final Verification

- ■ Verify Jenkins
- ■ Verify ECR
- ■ Verify ECS
- ■ Verify ALB
- ■ Verify frontend
- ■ Verify /api/* routing
- ■ Verify expected application result

End of README. Source scope note: the uploaded PDF ends at Step 6.4. No numbered Step 7 is present in the source.